

Integration of Classy-Blocks and OpenFOAM within Blender

B C H Benjamin

Week 10 Progress Report

OpenFOAM GUI

FOSSEE, IIT Bombay

May 16, 2026

- 1 Introduction
- 2 Research Gap
- 3 Problem Statement
- 4 Geometry Export & Pipeline Routing
- 5 Advanced Block Projection
- 6 OpenFOAM Environment Configuration
- 7 Tutorial Management System
- 8 Unified Pipeline & Auto-Update

Introduction

- ◀ **OpenFOAM:** A C++ toolbox for CFD using finite volume methods.
- ◀ **Classy-Blocks:** A Python library for procedural `blockMeshDict` generation, mimicking modern 3D CAD modeling.
- ◀ **Blender Integration:** Utilizing Blender as a frontend for geometry extraction and mesh visualization using `pyvista`.

Research Gap

- ◀ **Gap 1:** Lack of native Blender 5.x support for advanced Python meshing libraries like `classy_blocks`.
- ◀ **Gap 2:** Absence of an integrated “One-Click” solution for geometry extraction, mesh building, and quality reporting within a unified environment.

Problem Statement

- ◀ To develop a Blender Add-on that leverages `classy_blocks` and `pyvista` to automate OpenFOAM case setup, allowing users to extract geometry, generate high-quality meshes, and execute solvers without leaving the Blender interface.

Work Done: STL Extraction & Dynamic Routing

- ◀ **Terrain Export Operator:** Developed `MESH_OT_export_terrain_stl` to extract Blender's active geometric context (e.g. displaced planes) directly into the active OpenFOAM `constant/triSurface` directory.
- ◀ **Dynamic Spec Routing:** Upgraded `build_from_spec` to automatically dispatch geometry build logic. If a block specifies surface mappings, it seamlessly routes generation to the specialized `build_box_block_with_stl` function.

Work Done: Surface Conformity

- ▶ **STL Face Mapping:** Implemented direct integration with `classy_blocks` `project_side()` API. Users can select any local block face (e.g., top, bottom) and logically bind it to the exported TriSurface STL.
- ▶ **End-to-End Conformity:** Fully wired the feature set to ensure that when `blockMeshDict` is generated, the configured block faces automatically conform to the underlying complex terrain representations within OpenFOAM.
- ▶ **Rigorous Safety Tests:** Developed targeted test suites mocking the block builder safely without requiring mesh instantiation, validating side mappings without crashing out-of-bounds `classy_blocks` constraints.

Work Done: OpenFOAM Directory Configuration

- ◀ **Problem:** Blender does not always inherit terminal environment variables (like `$FOAM_RUN` and `$FOAM_TUTORIALS`) when launched from desktop shortcuts, leading to path resolution errors.
- ◀ **Solution:** Implemented a standalone configuration panel (`addon/foam_directories.py`) allowing users to explicitly set their OpenFOAM environment directories.
- ◀ **Implementation Details & Auto-Detection:**
 - Implemented robust path auto-detection: checks environment variables first, then uses `getpass` to scan `~/OpenFOAM` for the user's run directory, and scans `/opt` or `/usr/lib` for OpenFOAM tutorials.
 - Auto-detected paths are dynamically injected into the UI as defaults, displaying an 'INFO' label if successful.

Work Done: Interactive Tutorial Manager

- ◀ **Dynamic Discovery:** Implemented `get_tutorial_items` to recursively scan the read-only `$FOAM_TUTORIALS` directory for valid OpenFOAM cases (folders containing `system/`). Uses pruning heuristics and a module-level dictionary cache to prevent UI freezing.
- ◀ **Copy Operator:** Developed `CLASSY_OT_copy_tutorial` utilizing `shutil.copytree` to duplicate selected tutorials into the user's `$FOAM_RUN` directory safely, avoiding overwrites.
- ◀ **Searchable UI:** Upgraded the standard dropdown to a Search Operator (`CLASSY_OT_search_tutorials`) utilizing Blender's `invoke_search_popup`. This allows users to use keyboard inputs to rapidly filter through hundreds of OpenFOAM tutorials instantly.

Work Done: Week 10 Additions

- ◀ **Unified Directory Validation:** Developed a pure-Python validation module (`foam_path_utils.py`) to ensure all operations execute within the active `$FOAM_RUN` environment, including pre-emptive write permission checks and symlink resolution.
- ◀ **Depsgraph Auto-Update:** Implemented a `bpy.app.timers` and `depsgraph` handler to enable a live-meshing workflow. Changes to the scene automatically trigger the pipeline after a configurable debounce delay.
- ◀ **Dynamic Bashrc Resolution:** Refactored `foam_runner.py` to unify OpenFOAM environment sourcing, replacing hardcoded paths with a dynamic fallback system linked to Addon Preferences.
- ◀ **Test Suite Expansion:** Expanded the test suite to 71 passing tests, fully covering the new directory lifecycle, path validation, and tutorial copying logic.